Architectural Blueprint and Generative Orchestration Plan for Dungeon Haul

1. Executive Summary and Strategic Orchestration Framework

The translation of the *Dungeon Haul* Game Design Document into a functional, highly performant browser-based multiplayer application represents a complex software engineering endeavor. The project is defined as a fast-paced, four-player local drop-in/drop-out side-scrolling platformer requiring intricate physics, state synchronization, and dynamic scoring systems[1]. Players navigate a dungeon, collect randomized treasure, navigate traps, and compete to exit with the highest haul, which is subjected to a complex algorithmic "Share Modifier" system[1]. Deploying this architecture over the web necessitates the fusion of a modern Python backend utilizing the FastAPI framework, managed by the uv package ecosystem, with a WebGL-accelerated frontend built upon the Phaser 3 game engine[2].

To construct this architecture entirely through Large Language Models and generative AI agents, a rigorous, phased prompting methodology must be established. Generative agents, while capable of vast code synthesis, suffer from context degradation, hallucination of non-existent application programming interfaces, and architectural drift if left unconstrained. Therefore, the orchestration plan strictly divides the development lifecycle into isolated, heavily contextualized prompt sequences. Each prompt is designed to constrain the artificial intelligence within specific domain boundaries, ensuring that frontend logic successfully handles hardware acceleration degradation[2], while backend routing avoids execution anomalies[2].

This report provides an exhaustive, step-by-step master plan for utilizing generative systems to construct both the software infrastructure and the multimedia graphical and acoustic assets required for *Dungeon Haul*.

2. Foundational Artificial Intelligence Prompting Methodology

The successful deployment of autonomous agents to build a full-scale multiplayer game relies on a structural paradigm known as contextual scaffolding. The orchestrator must not issue a monolithic command to build the entire application simultaneously. Instead, the agent must be guided through a sequential pipeline where the verified outputs of one phase become the immutable context for the next phase.

Modern coding agents operate optimally when provided with a system prompt defining their persona, a strict set of constraints, and the minimum necessary context to achieve the immediate task. Before executing any code generation, an initial project initialization prompt must establish the global environment. The foundational system directive must be established

for the primary coding agent. The orchestrator must instruct the agent to adopt the persona of a senior Full-Stack Game Architect specializing in Phaser 3 and FastAPI, prioritizing deterministic, modular architecture. The agent must be instructed to rely exclusively on modern standard libraries and explicitly permitted plugins, outputting only raw code blocks and filesystem paths without conversational filler.

The logic and rules of the game must be extracted from the design documentation and fed to the agent dynamically to prevent logic hallucinations. The original premise dictates that players navigate traps, collect treasure, and compete for shares based on an arcane series of rules[1]. The overarching rule logic must be summarized into a strict data structure that is appended to the context window of every subsequent prompt. The structural constraints that must be provided to the AI are detailed in the following table.

Context Element	Data Structure Mandate	Architectural Rationale
Player Variables	SPEED, JUMP_HEIGHT, WEIGHT_PENALTY	Ensures the agent utilizes a centralized physics configuration rather than hardcoding arbitrary numerical values across different rendering and logic files[1].
Entity Dimensions	Base pixel constraints for entities	Prevents the agent from creating mismatching bounding boxes that break spatial hashing collision logic[2].
Game States	Idle, Instructions, Game, Fork, Results	Forces the AI to utilize standard Phaser Scene routing, preventing memory leaks caused by rendering overlapping execution loops[1].

3. Phase 1: Backend Infrastructure and Environment Initialization

The deployment architecture mandates a serverless orchestration on Google Cloud Run, backed by FastAPI and managed by the Rust-based uv package manager[2]. This specific stack requires meticulous prompting to ensure deterministic dependency resolution and correct routing protocols.

3.1 Prompt Sequence 1: Package Management and File Structure

The first task delegated to the coding agent is the initialization of the backend workspace. Historically, Python engineering suffered from bloated environments where dependencies were inherited transitively without intentional definition[2]. The integration of the uv ecosystem introduces deterministic lockfiles, cryptographically hashing every dependency to guarantee identical software environments across local machines and production containers[2].

The orchestrator must prompt the agent to generate the pyproject.toml, the Dockerfile, and the directory structure. The prompt must strictly constrain the output to utilize a multi-stage execution pattern designed to maximize cache retention. The agent must be instructed to select the python:3.13-slim image, as Alpine variants rely on the musl C standard library, which frequently breaks compatibility with Python packages requiring pre-compiled C-extensions[2]. Furthermore, the agent must use the --from flag to securely copy the compiled Rust binary directly from Astral's official container registry, preventing the pollution of the final container layer with unnecessary download utilities[2]. The agent must execute uv sync --frozen immediately after copying the lockfile to isolate dependency caching from the rapidly iterating game source code[2].

By forcing the artificial intelligence to generate the Dockerfile with the frozen synchronization command, the deployment pipeline is shielded from transitive dependency mutations. If the agent were unconstrained, it would default to legacy installation methods, resulting in non-deterministic builds and exponentially longer continuous integration compilation times[2].

3.2 Prompt Sequence 2: FastAPI Routing and Single Page Application Fallback

The backend must act simultaneously as a stateless application programming interface for game state synchronization and a static file server for the WebGL payload. Serving a browser game requires instructing FastAPI to act as a static file server by mounting the StaticFiles class to a specific uniform resource locator path[2].

A critical architectural anomaly arises when implementing client-side routing. If the catch-all route is declared before the application's interface endpoints, Starlette's sequential route matching architecture will eagerly intercept all calls, silently returning the hyper-text markup language content with a successful status code and catastrophically crashing the client's parser[2].

The orchestrator must prompt the AI to write the main execution script, strictly defining the endpoint structure. The agent must implement a health check endpoint and a mock score submission endpoint. Crucially, the prompt must contain an explicit, non-negotiable directive that the catch-all single page application fallback function must be declared absolutely last in the file execution order[2]. This constraint guarantees that the API intercepts legitimate backend requests before the routing tree hands the request over to the static directory.

4. Phase 2: Frontend Engine Initialization and Viewport

Scaling

Browser fragmentation requires the game canvas to adapt to arbitrary aspect ratios ranging from ultra-wide desktop monitors to portrait-oriented cellular phones[2]. The design must dictate how the WebGL canvas reacts. Permitting the camera's field-of-view to expand dynamically grants players on ultra-wide monitors an unfair competitive advantage, as they can perceive environmental hazards and enemy spawn vectors earlier than constrained players[2].

4.1 Prompt Sequence 3: The Phaser 3 Scale Manager

The orchestrator must direct the artificial intelligence to configure the Phaser 3 initialization object to handle responsive scaling via strict letterboxing. The agent must generate the base configuration file setting the rendering type to WebGL, with a graceful degradation to the Canvas application programming interface[2].

The prompt must explicitly forbid the use of the EXACT_FIT scaling mode. Evidence indicates that EXACT_FIT allows the canvas to automatically resize to the window without respecting the original pixel ratio, causing severe vertical stretching of the pixels when the window shrinks[6]. Instead, the AI must be commanded to utilize Phaser.Scale.FIT combined with Phaser.Scale.CENTER_BOTH. This mode combination automatically adjusts the width and height to fit inside the given target area while strictly keeping the original aspect ratio, calculating the appropriate margins to center the game canvas vertically and horizontally[3].

The agent must be instructed to define a fixed internal logical resolution, disable anti-aliasing, and initialize the Arcade Physics subsystem. The implementation of this exact scaling methodology ensures that the digital artwork remains perfectly proportioned regardless of the host hardware[3].

5. Phase 3: Generative Asset Pipeline Engineering

Creating the graphical and acoustic assets requires transitioning away from coding agents toward specialized generative models. The constraints of a browser-based game jam project necessitate severe memory and bandwidth optimizations[2]. Delivering hundreds of individual image files results in connection pooling bottlenecks, so all assets must be algorithmically packed into optimal formats[2]. The prompts must be engineered to ensure stylistic cohesion, orthographic consistency, and technical compatibility.

5.1 Visual Asset Generation via Diffusion Models

The design documentation specifies four primary playable characters: the Gnome, the Sprite, the Halfling, and the Dwarf[1]. It also requires diverse environmental blocks corresponding to specific biomes, including the Gold Hoard, Green Outside, Grey Dungeon, Red Lava, Blue Ice, Brown Cavern, and Purple Mist[1]. Generative diffusion models struggle to output pre-formatted animation sprite sheets natively. Therefore, the workflow must focus on generating isolated, high-fidelity assets, utilizing a chroma key methodology, and subsequently processing them.

The orchestrator must utilize specialized prompt formulas for the image generation agent (e.g., Midjourney). The prompts must explicitly demand a two-dimensional side-scrolling

perspective, preventing the AI from generating isometric or three-quarter perspectives that break the platforming illusion. The prompt must dictate a pure green background to facilitate perfect automated background removal.
For character generation, the prompt structure must request a specific character class carrying a stylized sack of treasure, rendered in a fantasy pixel art style with a constrained 16-bit color palette. The aspect ratio must be locked to a square proportion to ensure uniform bounding box calculations during the physics implementation. For environmental tilesets, the prompt must request seamless, tileable textures matching the specific elemental themes defined in the documentation[1].

5.2 Algorithm Asset Packing and Optimization

Once the raw graphical assets are generated and the backgrounds removed via automated alpha-masking utilities, the assets must be aggregated. The orchestrator must return to the primary coding agent to construct an ingestion pipeline.
The coding AI must be prompted to write a specialized script utilizing the Python Pillow library. The script must ingest a directory of transparent image frames and pack them into a MaxRects-optimized Sprite Atlas[2]. The prompt must dictate that the final atlas dimensions are strictly constrained to Power-of-Two measurements, a mathematical requirement highly preferred by graphical processing unit memory allocators to prevent rendering artifacts[2]. Furthermore, the script must calculate the precise coordinates of each packed sprite and generate an accompanying JavaScript Object Notation map. The agent must be instructed to output the final graphic utilizing the modern WebP compression format, significantly reducing payload weight while preserving the essential alpha channels[2].

5.3 Acoustic Asset Generation and Synthesis

Audio compression is a critical vector for bandwidth optimization. The architecture specifies that background compositions utilize the Ogg Vorbis format for sample-accurate looping capabilities, while short sound effects utilize uncompressed immediate formats to prevent decoding latency[2]. The audio landscape of the game includes character sounds, object impacts, trap mechanisms, and interface notifications[1].
The orchestrator must feed targeted prompts into generative audio models. For the background acoustics, the prompt must request instrumental, chiptune fantasy dungeon crawling music characterized by a stealthy, steady rhythmic pulse. The prompt must dictate a low-pass filter to establish thematic tension without causing auditory fatigue[2].
For sound effects, the prompts must target specific psychoacoustic properties. The documentation lists precise required effects, such as a crumbling block breaking, a lightning trap zapping, a golem stomping, and interface confirmation tones[1]. The generative audio prompts must request high-frequency bursts for interface confirmations, and bass-heavy digital fragmentation sounds for the destruction of heavy stone blocks[1].

6. Phase 4: Mechanics, Input State, and Drop-In Multiplayer

The defining mechanical characteristic of *Dungeon Haul* is its local, four-player drop-in and drop-out capability. The control scheme is originally modeled after a classic Nintendo Entertainment System controller, requiring directional inputs alongside dedicated jump, trip, push, drop, and throw actions[1]. Orchestrating inputs from multiple gamepads and a shared keyboard requires intercepting native document object model events and routing them through a unified abstraction layer.

6.1 Prompt Sequence 4: Unified Input Manager Implementation

Native input polling within the framework is insufficient for complex local multiplayer scenarios where devices must dynamically map to available character slots. The orchestrator must direct the artificial intelligence to utilize a specialized plugin, specifically the phaser3-merged-input ecosystem, to amalgamate peripheral data[9].

The AI must be prompted to implement a centralized Input Manager class. The constraints must explicitly dictate the creation of four distinct player object references[9]. The prompt must instruct the agent to bind specific keyboard arrays to the first two player slots, such as mapping the W, A, S, D keys to Player 1, and the arrow keys to Player 2[7]. Simultaneously, the agent must be instructed to bind connected gamepads to the remaining unassigned players. The prompt must explicitly command the AI to utilize mapped button names (e.g., RC_S for the primary action button) rather than relying on arbitrary integer mappings that vary across hardware manufacturers[9]. The final constraint for this prompt requires the agent to expose unified getter methods that evaluate both the continuous isDown state and the boolean isJustDown state across all bound peripherals, resolving historical architectural conflicts between polling and event callbacks[5].

6.2 Prompt Sequence 5: Drop-In Artificial Intelligence State Machine

The design documentation strictly requires that the game environment always features exactly four active characters to maintain dynamic pacing[1]. If a human player abandons their peripheral, an artificial intelligence controller must seamlessly assume control of the avatar. The coding agent must be prompted to create a state machine controller class that toggles between a human state and an autonomous state. The exact logic from the documentation must be fed into the prompt: The agent must track a timestamp of the last valid input. If the temporal difference exceeds twenty seconds, the controller must instantly transition to the autonomous state[1]. Furthermore, if the entity's bounding box intersects the edge of the camera viewport and no input has been detected for five seconds, the transition must also trigger[1]. Any subsequent registered input event from the assigned peripheral must immediately revert the state to human control[1].

The behavior of the autonomous controller must also be explicitly defined in the prompt. During the autonomous state, the agent must instruct the entity to query the scene graph for all currently active human-controlled entities. The entity must calculate the geometric average position of all humans and apply a velocity vector towards that centralized point, tolerating a predefined distance gap[1]. For treasure acquisition, the autonomous entity must identify nearby items and move toward them, provided it is not carrying a quantity of treasure exceeding the

highest amount carried by any active human player[1]. By separating the controller logic from the rendering logic in the prompt, the agent guarantees that swapping paradigms does not require destroying and instantiating a new graphical sprite, thereby reducing garbage collection overhead in the browser.

7. Phase 5: Physics Step, Collision Resolution, and Delta-Time Interpolation

The core execution loop relies on real-time physics, necessitating a tight coupling between the user state and frame rendering[2]. Browser environments present a unique threat to game physics: tab suspension. When a user switches browser tabs, the environment aggressively throttles or entirely suspends the animation frame loop. Upon return, the calculated time elapsed since the last frame becomes massive. If the artificial intelligence naively multiplies entity velocity by this delta, characters will quantum-tunnel through solid geometry, irrevocably breaking the collision state[2].

7.1 Prompt Sequence 6: Fixed-Timestep Accumulation

The orchestrator must prompt the agent to write the primary update loop for the active gameplay state. The prompt must explicitly forbid naive delta-time multiplication. Instead, the agent must implement a fixed-timestep accumulator logic. The constraint must dictate that the maximum allowable delta is capped at a strict threshold, forcing the engine to execute multiple smaller physics steps to catch up without sacrificing collision integrity[2].

The prompt must also detail the mass and velocity logic. The agent must update character velocities based on carried weight. The prompt must define a baseline speed parameter and instruct the agent to mathematically deduct a calculated percentage for every piece of treasure carried beyond the initial three items[1]. To optimize performance, the agent must be instructed to utilize spatial partitioning methodologies, such as quadtrees, to validate intersections between the four players, hundreds of potential dropped treasures, and autonomous hostile entities, reducing computational complexity from exponential to logarithmic scales[1].

7.2 Prompt Sequence 7: Trap Actuation and Entity Interactivity

The environment is hostile, populated by traps that dynamically alter the physics state of the players. The documentation details multiple variants: Spikes, Crumbling Bricks, Receding Blocks, Cycling Lightning, Gas releases, and Falling Rocks[1]. Furthermore, entities such as Golems and Phantom Hands hunt the players[1].

The orchestrator must prompt the agent to build a base trap class that inherits from the engine's sprite object[4]. The prompt must dictate that upon an overlap intersection trigger between a player and a trap, the player's controller is temporarily disabled and forced into a stunned animation matrix[1]. During this execution frame, the agent must iterate through the player's carried treasure stack. The prompt must instruct the AI to detach the top objects from the player, instantiate active physics bodies for them, and apply random upward and outward velocity vectors to simulate kinetic spillage[1]. Crucially, the prompt must demand a temporal

lockout timer on the dropped items, preventing the original owner from re-acquiring the items for a defined fraction of a second, intentionally allowing rival players the opportunity to steal the loose haul[1].

8. Phase 6: Environmental Rendering and Parallax Architecture

Levels are composed of a strict grid of square blocks[1]. Rather than attempting to prompt the AI to hardcode coordinate arrays for nineteen different sequential levels, the orchestration plan dictates the creation of a generalized parsing engine.

8.1 Prompt Sequence 8: Tilemap Parser and Material Properties

The coding agent must be prompted to develop a level ingestion system that reads a structured grid array and maps the coordinate data to specific material classes. The prompt must include the behavioral constraints of the decorative and structural blocks. The agent must program the ice surface class to dynamically reduce the deceleration friction property of any intersecting player, simulating a slippery environment[1]. Conversely, the sand surface class must be programmed to increase friction and decrease the maximum velocity bounds[1]. Heavy switches must be programmed to require a threshold calculation, only activating if the total mass of the intersecting players and their carried treasure exceeds a predefined limit[1].

The environment relies on visual depth to communicate atmosphere. The prompt must instruct the agent to instantiate multiple distinct rendering layers. The agent must map the Far Background layer to scroll at a reduced rate of fifty percent relative to the camera, the Near Background at parity with the camera, the Midground as the primary interactive space, and the Foreground at an accelerated rate of one hundred and twenty-five percent to create an aggressive parallax illusion[1].

8.2 Prompt Sequence 9: The Path Fork Discussion Mechanic

At the termination of an active level, the design requires the game state to transition to an isolated fork screen where players determine the subsequent route[1]. This acts as a pseudo-democratic voting mechanic disguised as frantic gameplay.

The orchestrator must prompt the agent to instantiate the fork state class. The constraints must explicitly define the interaction loop. The agent must render two distinct exit gateways themed after the subsequent biomes. The players utilize their directional inputs to select a path. Once a path is highlighted, the agent must capture rapid actuations of the primary action buttons, classifying them as arguments[1]. The prompt must instruct the agent to instantiate a counter for each path that increments per valid button press. After a predetermined temporal limit, the agent must mathematically compare the arrays, identify the victor, and trigger the scene manager to ingest and render the corresponding level identifier[1].

9. Phase 7: Scoring Orchestration and Cinematics

The climax of a session involves the complex, algorithmic distribution of the accumulated wealth. Players are evaluated not just on the raw value of their haul, but on behavioral statistics

gathered throughout the entire seven-level run. The final payout is heavily modified by a system of rewards and penalties[1].

9.1 Prompt Sequence 10: The Share Modifier Algorithm

The rules require tracking highly granular metrics. The agent must be prompted to construct an overarching score orchestrator singleton class that persists across scene destruction. To prevent the artificial intelligence from hallucinating incorrect logic, the full table of share modifiers must be provided in the prompt constraints.

Share Modifier Title	Type	Algorithmic Condition	Share Value
Leader of the Pack	Reward	Boolean flag for the first entity to trigger the exit coordinate array across all levels[1].	+10
Breadwinner	Reward	Highest integer value of gross recovered treasure[1].	+5
Airhead	Reward	Highest accumulated frame count with a negative Y-velocity[1].	+3
Flawless	Reward	Boolean flag for zero registered stun or hurt states throughout the entire session[1].	+5
Gambler	Reward	Array evaluation: Inventory contains exclusively Chest entities[1].	+5
Opportunist	Reward	Final inventory integer is at least +2	+2

		greater than initial hoard exit integer[1].	
Big Jerk	Penalty	Highest integer count of collision overlaps resulting in a hostile trip/push event[1].	-5
Klutz	Penalty	Highest integer count of trap overlap triggers[1].	-3
Empty Handed	Penalty	Final inventory integer equals zero[1].	-3
Autopilot	Penalty	Controller state evaluated as autonomous artificial intelligence for greater than fifty percent of the total session duration[1].	-5

Because the share modifiers are conditionally awarded based on relative comparisons between the four concurrent players (e.g., hit the most traps), the prompt must explicitly instruct the agent to calculate absolute totals for all entities simultaneously. The agent must then execute a sorting algorithm across the arrays to assign the specific modifier flags before executing the final geometric payout formula: the player's specific take is equal to the total group treasure multiplied by their specific individual share quotient divided by the total sum of all distributed shares[1].

9.2 Prompt Sequence 11: The End Screen Cinematic Rendering

The final visual component requires precise timing and animation. The agent must be prompted to script the cinematic sequence for the results screen. The prompt must command the agent to order the character execution from the slowest historical stage exit time to the fastest[1]. The agent must script a tweening animation utilizing the engine's built-in mathematical curves to visually toss each inventory item from the character into a central coordinate pile[1]. The prompt must dictate a continuous evaluation of the treasure array during this animation. If an item entering the pile completes a predefined set mathematically (such as the four pieces of the Suit of Armor, or the three Celestial Markers), the agent must trigger an interface pop-out

displaying the multiplicative completion bonus[1]. Following the pile accumulation, the agent must be prompted to render stylized panels behind each character, sequentially listing their earned Share Modifier titles, strictly color-coded: gold for unique rewards, white for common rewards, blue for common penalties, and red for unique penalties[1].

10. Phase 8: Acoustic Implementation and Engine Integration

Browser implementations of the Web Audio application programming interface are notoriously idiosyncratic. Operating systems frequently require explicit, registered user interaction before the browser security policy permits the decoding or synthesis of sound[2]. A failure to account for this architecture results in a silent application and an console error log.

10.1 Prompt Sequence 12: Audio Context Orchestration

The orchestrator must prompt the artificial intelligence to build an audio manager class that respects these browser constraints. The prompt must explicitly forbid the agent from attempting to execute playback functions immediately upon the initial boot sequence. Instead, the agent must bind the audio context resumption function to a global event listener awaiting the first valid keystroke or gamepad actuation during the idle title screen state[2].

The prompt must instruct the agent to differentiate audio routing based on the file format. The background environmental tracks, utilizing the compressed Ogg Vorbis format, must be routed through spatial panning nodes, allowing the engine to mathematically interpolate the volume down during interface states such as the fork discussion[1]. Conversely, the uncompressed sound effects must be allocated to an array of highly available, concurrent audio channels, ensuring that high-frequency events, such as multiple autonomous enemies triggering traps simultaneously, do not cause audio clipping or dropouts due to channel exhaustion[2].

11. Phase 9: Containerization, Testing, and Infrastructure Deployment

With the entire codebase synthesized, the final phase utilizes the generative systems to script the continuous integration pipeline and execute the deployment to the Google Cloud Run infrastructure[2]. This serverless platform possesses the elasticity to instantly scale out to handle massive concurrent traffic spikes, while scaling down to absolute zero instances during idle periods, ensuring zero financial cost for dormant game jam projects[2].

11.1 Prompt Sequence 13: The Cloud Run Execution Pipeline

The final deployment requires securely moving the immutable Docker container artifact into the remote cloud environment. The orchestrator must prompt the artificial intelligence to write the necessary Google Cloud software development kit bash execution script.

The prompt must require the agent to generate commands that initialize an Artifact Registry repository[12]. Subsequently, the agent must generate the cloud build submission command, taking the optimized, multi-stage Dockerfile constructed in Phase 1 and passing it to the

remote compiling servers[13]. The final deployment command generated by the AI must include the flag to permit unauthenticated invocation. Because the browser game must be accessible to the general internet public without identity verification, this flag is strictly required to permit unauthenticated requests to download the static payload[12]. Furthermore, the agent must configure the command to inject the default port binding variable into the container, mapping the traffic to the internal FastAPI ASGI server[2].

Serverless platforms operate on specialized sandbox engines utilizing highly ephemeral filesystems. Any data written to the local disk memory of the instance will be permanently obliterated when the instance is spun down[2]. The orchestrator must anticipate this limitation. The final prompt must mandate that the artificial intelligence connect the previously stubbed high score application programming interface endpoint to an external persistence layer, utilizing the native cloud secret management bindings to inject necessary database keys directly into the environment variables at runtime[2].

12. Synthesis and Architectural Conclusion

By systematically orchestrating generative artificial intelligence models through this deeply isolated, heavily constrained phased prompting structure, the inherent risks of autonomous hallucinations and context collapse are entirely mitigated. The deployment architecture successfully bridges the incredibly strict physics and networking requirements of a fast-paced, four-player side-scrolling browser game[1] with the modern rigors of scalable, deterministic, containerized Python backends[2].

The forced utilization of specialized libraries like phaser3-merged-input resolves the complexities of unified input polling and peripheral amalgamation[9], while the exact application of the Phaser.Scale.FIT mechanism permanently prevents graphical texture warping across disparate client screens[3]. Following these discrete, highly technical prompt sequences ensures the transition from the theoretical *Dungeon Haul* Game Design Document to a functional, globally deployable software artifact.

Works cited

1. TOJam 8_ Dungeon Haul Design Document.pdf
2. Dungeon Haul Game Design Document Analysis.pdf
3. Phaser.Scale | Phaser Help, https://docs.phaser.io/api-documentation/namespace/scale
4. [SOLUTION] Scaling for multiple devices,resolution and screens - HTML5 Game Devs Forum, https://www.html5gamedevs.com/topic/5949-solution-scaling-for-multiple-devicesresolution-and-screens/
5. Handling Inputs in Phaser 3: Part 2: Gaining Control Over Controllers | Coding into the Void, https://blog.khutchins.com/posts/phaser-3-inputs-2/
6. Phaser: Scaling to fill screen, maintain pixel ratio, but not aspect ratio, https://gamedev.stackexchange.com/questions/143267/phaser-scaling-to-fill-screen-maintain-pixel-ratio-but-not-aspect-ratio

7. "Making your first Phaser 3 game" two local players., https://phaser.discourse.group/t/making-your-first-phaser-3-game-two-local-players/1842
8. Scale manager - Notes of Phaser 4 - GitHub Pages, https://rexrainbow.github.io/phaser3-rex-notes/docs/site/scalemanager/
9. GaryStanton/phaser3-merged-input: A Phaser 3 plugin to map input from keyboard & gamepad to player actions - GitHub, https://github.com/GaryStanton/phaser3-merged-input
10. Issue when holding keyboard input as the scene restart - Phaser 3, https://phaser.discourse.group/t/issue-when-holding-keyboard-input-as-the-scene-restart/12540
11. Handling Inputs in Phaser 3: Part 1: Humble Beginnings | Coding into the Void, https://blog.khutchins.com/posts/phaser-3-inputs-1/
12. [Solved] Phaser 3 Game Scaling (Letterbox scale), https://phaser.discourse.group/t/solved-phaser-3-game-scaling-letterbox-scale/677
13. Phaser 3 API Documentation - Class: ScaleManager - GitHub Pages, https://photonstorm.github.io/phaser3-docs/Phaser.Scale.ScaleManager.html